

11 MVC deel 3 - SelectListItem & Identity in MVC

📅 09-09-2026 ⌚ Ongeveer 16 minuten 🗨️ Ongeveer 2446 woorden

In het vorig hoofdstuk leerde je hoe je formulieren maakt voor het aanmaken, wijzigen en verwijderen van gegevens uit een database. Je gebruikte ViewModels om gegevens door te geven van je Controller naar je View en omgekeerd. Dit hoofdstuk start met een uitbreiding op deze formulieren. Er wordt uitgelegd hoe je een **SelectList** maakt om een lijst van gegevens te tonen en het geselecteerde item uit de lijst terug te geven aan de Controller. In het tweede deel van de cursus wordt uitgelegd hoe je **Identity** implementeert in MVC. Je maakt een UserInterface om een gebruiker te registreren, aan te melden, uit te loggen en om gebruikers te beheren. De backend voor een GebruikersController wordt beschreven in hoofdstukken 6 en 7.

Setup

Clone het project MVCDemo.

SelectItems in formulieren

In het volgende voorbeeld wordt het aanmaken van een bestelling uitgebreid besproken. Op een nieuwe bestelling wil je een klant kiezen uit een lijst van klanten. Daarna wil je graag producten toevoegen aan de bestelling. In het voorbeeld kun je dynamisch orderlijnen toevoegen met Javascript. Je weet op voorhand niet hoeveel producten een klant wil bestellen. Het gedeelte van Javascript wordt niet verder besproken in deze cursus.

Het ViewModel voor het aanmaken van een bestelling heeft dus volgende gegevens: een lijst van klanten (om te tonen in de View), een lijst van producten (om te tonen in de View), een lijst van orderlijnen (welke orderlijnen moeten aan de bestelling toegevoegd worden) en een KlantId (van wie is de bestelling). Voor de orderlijnen maak je ook een apart ViewModel.

In dit voorbeeld wordt geen AutoMapper gebruikt om het ViewModel op te vullen. AutoMapper heeft namelijk geen ingebouwde ondersteuning voor SelectList.

Om een complexe oefening als dit uit te werken heb je volgende zaken nodig:

- 2 ViewModels (BestellingCreateViewModel en OrderlijnCreateViewModel)
- 2 Acties in de BestellingController (De eerste vult de SelectList op en toont de View, de tweede verwerkt de gegevens van het formulier)
- 1 View met het formulier

1. Maak het ViewModel `BestellingCreateViewModel` aan

```
1 public class BestellingCreateViewModel CS
2 {
3     public int KlantId { get; set; }
4
5     public List<SelectListItem> Klanten { get; set; } = new
6     List<SelectListItem>();
7     public List<OrderLijnCreateViewModel> OrderLijnen { get; set; } = new
8     List<OrderLijnCreateViewModel>();
9     public List<SelectListItem> Producten { get; set; } = new
10    List<SelectListItem>();
11 }
```

2. Maak het ViewModel `OrderlijnCreateViewModel` aan

```
1 public class OrderLijnCreateViewModel CS
2 {
3     public int ProductId { get; set; }
4     public int Aantal { get; set; }
5 }
```

3. Maak een actie `AddBestelling` in de `BestellingController` om het

`BestellingCreateViewModel` op te vullen. Je haalt eerst de klanten op en vult de `SelectList` op van klanten in het ViewModel. Elke `SelectList` heeft twee argumenten nodig: de `Id` die toegevoegd wordt in de database (`Value`) en een `Naam` die getoond wordt in de dropdown (`Text`). Daarna gebeurt hetzelfde voor producten. Daarna geef je het ViewModel mee met de View. `KlantId` en de orderlijnen zijn nu nog leeg.

```
1 public async Task<IActionResult> AddBestelling() CS
2 {
3     var viewModel = new BestellingCreateViewModel();
4
5     // Load customers into the dropdown
6     var klanten = await _context.KlantRepository.GetAllAsync();
7     viewModel.Klanten = klanten.Select(k => new SelectListItem
8     {
9         Value = k.Id,
10        Text = k.Naam
11    });
12 }
```

```

8         Value = k.Id.ToString(),
9         Text = k.Naam
10    }).ToList();
11
12    // Load products into the dropdowns for order lines
13    var producten = await _context.ProductRepository.GetAllAsync();
14    viewModel.Producten = producten.Select(p => new SelectListItem
15    {
16        Value = p.Id.ToString(),
17        Text = p.Naam
18    })
19    .ToList();
20    return View(viewModel);
21 }
22

```

4. Maak een actie AddBestelling met het kenmerk **[HttpPost]** in de `BestellingController` om het `BestellingCreateViewModel` in te lezen en te verwerken. De verwerking gebeurt als volgt: als de formulier correct is doorgegeven, wordt er een nieuwe bestelling aangemaakt en worden de orderlijnen aan de bestelling toegevoegd. Daarna worden de wijzingen doorgevoerd naar de database.

```

1  [HttpPost]
2  public async Task<IActionResult> AddBestelling(BestellingCreateViewModel
3  viewModel)
4  {
5      if (ModelState.IsValid)
6      {
7          var bestelling = new Bestelling
8          {
9              KlantId = viewModel.KlantId,
10             Orderlijnen = new List<Orderlijn>()
11         };
12
13         foreach (var orderLijnViewModel in viewModel.OrderLijnen)
14         {
15             var orderlijn = new Orderlijn
16             {
17                 ProductId = orderLijnViewModel.ProductId,
18                 Aantal = orderLijnViewModel.Aantal
19             };
20             bestelling.Orderlijnen.Add(orderlijn);
21         }
22

```

```

22
23         await _context.BestellingRepository.AddAsync(bestelling);
24         _context.SaveChanges();
25
26         return RedirectToAction("Index");
27     }
28     return View(viewModel);
}

```

5. Ontwerp de View met de zelfde naam als de actie: `AddBestelling`. Het JS-script, voor het toevoegen van orderlijnen, valt buiten de scope van deze cursus. Wat wel belangrijke leerstof is, is het **Select-element**. Je ziet dat de tag-helper *asp-for* toegewezen is aan `KlantId`. Dit is de `Id` van de klant die overeenkomt met de gekozen waarde uit *asp-items*. Hetzelfde geldt voor de producten in een orderlijn.

```

1      @model BestellingCreateViewModel                                html
2      @{
3          ViewBag.Title = "Addbestelling";
4      }
5      <h2>Nieuwe bestelling</h2>
6
7      <form asp-action="AddBestelling">
8          <div class="form-group">
9              <label asp-for="KlantId" class="control-label">Selecteer een
10             klant:</label>
11              <select asp-for="KlantId" class="form-control" asp-
12             items="Klanten"></select>
13          </div>
14          <div></div>
15          <h4>Orderlijnen:</h4>
16          <table class="table">
17              <thead>
18                  <tr>
19                      <th>Product</th>
20                      <th>Aantal</th>
21                      <th></th>
22                  </tr>
23              </thead>
24              <tbody id="orderLinesContainer">
25                  @for (var i = 0; i < Model.OrderLijnen.Count; i++)
26                  {
27                      <tr>

```

```

28         <td>
29             <select asp-for="OrderLijnen[i].ProductId"
30 class="form-control" asp-items="Model.Producten"></select>
31         </td>
32         <td>
33             <input type="number" asp-
34 for="OrderLijnen[i].Aantal" class="form-control" />
35             <span asp-validation-for="OrderLijnen[i].Aantal"
36 class="text-danger"></span>
37         </td>
38         <td>
39             @if (i > 0)
40             {
41                 <a href="#" class="btn btn-danger"
42 onclick="removeOrderLine(this)">Verwijder</a>
43             }
44         </td>
45     </tr>
46 }
47 </tbody>
48 </table>

    <div class="form-group">
        <button type="button" class="btn btn-primary"
onclick="addOrderLine()">Voeg product toe</button>
        <input type="submit" value="Bestelling toevoegen" class="btn btn-
success" />
    </div>
</form>

```

Het editeren van een bestelling werkt op een gelijkaardige manier. Denk er aan dat je de id van de bestelling ook meegeeft in een **hidden**-element.

User Interface voor Identity

In hoofdstuk 6 en 7 werd Identity al uitgebreid besproken in verband met WebAPI. Hier werk je steeds met twee applicaties: de WebAPI en een clientapplicatie. De beveiliging gebeurt in een aantal stappen:

1. De gebruiker stuurt een GET-request vanuit de ClientApplicatie met een gebruikersnaam en een wachtwoord.

2. De WebAPI stuurt een response met een token indien de gebruiker toegang heeft tot het systeem.
3. De gebruiker stuurt nieuwe requests met het token in de hoofding van de request.

Afhankelijk van de rechten van de gebruiker heb je toegang tot bepaalde endpoints.

Je weet ook dat Identity werkt met een eigen systeem voor het beheren van gebruikers (UserManager), rollen (RoleManager) en aan te melden (SignInManager).

In de inleiding van Hoofdstuk 9 staat dat je op twee manieren een MVC-applicatie kunt opbouwen: met backend of zonder. Als je een applicatie met alleen de frontend ontwikkelt (zonder Identity, EntityFramework, Repositories of UnitOfWork) Moet je dezelfde, bovenstaande werkwijze hanteren als voor een WebAPI.

Als je een applicatie maakt met frontend én backend, kun je werken zonder het token. Je werkt nog steeds met UserManager, RoleManager en SignInManager en de CRUD-methoden zijn dezelfde als bij WebAPI.

Wat er ook bijkomt is een volledige User Interface die bestaat uit Views voor:

- Login
- Register
- Profiel
- Logout
- Wachtwoord verificatie
- Gebruikersbeheer (index)
- 2-stapsverificatie
- Create van gebruikers en/of rollen
- Update van gebruikers en/of rollen
- Delete van gebruikers en/of rollen
- Details van gebruikers en/of rollen
- ...

Met de NuGet Package *Microsoft.AspNetCore.Identity.UI* kun je een gans membershipsysteem scaffolden die al die pagina's gaat aanmaken. Er is echter nog altijd veel configuratiewerk nodig om het systeem volledig naar jouw hand te zetten.

Om de werking van Identity in MVC goed te kunnen begrijpen wordt in deze cursus een basic gebruikersbeheer uitgewerkt, waar je nadien vrij mee aan de slag kunt. Je zult zien dat de werkwijze niet veel verschilt van de vorige hoofdstukken.

Login

In Hoofdstuk 6 is uitgelegd hoe je met behulp van seeding een default gebruiker en rollen kunt aanmaken. Om met een gebruiker aan te kunnen melden, heb je volgende zaken nodig:

1. Een ViewModel voor de Login-View
2. Een Login-View
3. Een actie in een GebruikerController om de View te starten.
4. Een hyperlink naar deze actie.
5. En een actie in een GebruikerController om de Login te verifiëren.
6. Maak een nieuw ViewModel `GebruikerLoginViewModel` met de properties `Gebruikersnaam` en `PassWord`

```
1 public class GebruikerLoginViewModel
2 {
3     public string Gebruikersnaam { get; set; }
4     public string Password { get; set; }
5 }
```

2. Maak een View in map `\Views\Gebruiker\` met als naam `Login`. Voor deze View is een aparte layout gemaakt die bewaard wordt in map `Views\Shared`. Op regel 13 is een validatieveld voorzien voor het model. Hier komen foutmeldingen als de gebruikersnaam niet bestaat of het wachtwoord is niet juist.

```
1 @model GebruikerLoginViewModel
2 @{
3     ViewData["Title"] = "Aanmelden";
4     Layout = "/Views/Shared/_LoginLayout.cshtml";
5 }
6
7 <div class="d-flex justify-content-center">
8     <div class="frame">
9         <div>
10             <section>
```

```

11         <form id="account" method="post">
12             <br />
13             <div asp-validation-summary="ModelOnly" class="text-
14 danger"></div>
15
16             <div class="form-floating">
17                 <input asp-for="Gebruikersnaam" class="form-
18 control" autocomplete="Gebruikersnaam" aria-required="true" />
19                 <label asp-for="Gebruikersnaam" class="form-
20 label">Gebruikersnaam</label>
21                 <span asp-validation-for="Gebruikersnaam"
22 class="text-danger"></span>
23             </div>
24             <br />
25
26             <div class="form-floating">
27                 <input asp-for="Password" type="password"
28 class="form-control" autocomplete="current-password" aria-required="true"
29 />
30                 <label asp-for="Password" class="form-
31 label">Wachtwoord</label>
32                 <span asp-validation-for="Password" class="text-
33 danger"></span>
34             </div>
35             <br />
36             <div>
37                 <button id="login-submit" type="submit"
38 class="purpleButton">Aanmelden</button>
39             </div>
40         </form>
41     </section>
42 </div>
43 </div>
44 </div>

```

3. In de GebruikersController worden de UserManager, RoleManager en SignInManager geïnjecteerd en is er een actie voorzien om de Login-View op te starten. Deze actie is vrij toegankelijk voor alle gebruikers.

```
1 [AllowAnonymous]
2 public ActionResult Login()
3 {
4     return View();
5 }
```

4. In de Navigatiebalk voorzie je een hyperlink naar de Login-pagina, Logout-pagina en de Index-Pagina van de gebruiker. Hiervoor kun je een aparte razor-pagina `_LoginPartial` maken in de map `Views\Shared`. Hier injecteer je de `SignInManager` en de `userManager`. Op die manier kun je gegevens van de aangemelde gebruiker ophalen. Op `_LoginPartial` kun je dan bepalen welke acties zichtbaar zijn als je aangemeld bent.

```
1 @using Microsoft.AspNetCore.Identity
2
3 @inject SignInManager<CustomUser> SignInManager
4 @inject UserManager<CustomUser> UserManager
5
6 <ul class="navbar-nav">
7     @if (SignInManager.IsSignedIn(User))
8     {
9         <li class="nav-item">
10             <a class="nav-link text-dark" asp-controller="Gebruiker" asp-
11 area="" asp-action="Index">Hello @UserManager.GetUserName(User)</a>
12         </li>
13         <li class="nav-item">
14             <a class="nav-link text-dark" asp-controller="Gebruiker" asp-
15 area="" asp-action="Logout">Logout</a>
16         </li>
17     }
18     else
19     {
20         <li class="nav-item">
21             <a class="nav-link text-dark" asp-area="" asp-
22 controller="Gebruiker" asp-action="Login">Login</a>
23         </li>
24     }
25 </ul>
```

De `_LoginPartial` wordt toegevoegd aan de navbar in `_Layout`. (regel 10)

```

1 <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between"> CS
2
3     <ul class="navbar-nav flex-grow-1">
4         <li class="nav-item">
5             <a class="nav-link text-dark" asp-area="" asp-
6 controller="Home" asp-action="Index">Home</a>
7         </li>
8         <li class="nav-item">
9             <a class="nav-link text-dark" asp-area="" asp-
10 controller="Home" asp-action="Privacy">Privacy</a>
11         </li>
12     </ul>
13     <partial name="_LoginPartial" />
14 </div>

```

5. In de `GebruikerController` komt een actie om de gebruikersnaam en wachtwoord te verifiëren en vast te leggen dat de gebruiker aangemeld is (regel 21). Als er foutmeldingen zijn in het model, voeg je de foutmelding toe aan het model en geef je het model terug aan de Login-View. Als de gebruiker aangemeld is, wordt de startpagina geladen.

```

1 [HttpPost] CS
2 [AllowAnonymous]
3 public async Task<IActionResult> Login(GebruikerLoginViewModel model)
4 {
5     if (!ModelState.IsValid)
6         return View(model);
7
8     var user = await _userManager.FindByEmailAsync(model.Email);
9     if (user != null && !user.EmailConfirmed)
10     {
11         ModelState.AddModelError("", "Emailadres is nog niet
12 bevestigd.");
13         return View(model);
14     }
15     if (await _userManager.CheckPasswordAsync(user, model.Password) ==
16 false)
17     {
18         // Nooit exacte informatie geven: zeg alleen dat combinatie
19 verkeerd is...
20         ModelState.AddModelError("", "Verkeerde logincombinatie!");
21         return View(model);
22     }
23 }

```

```
22     }
23
24     var result = await _signInManager.PasswordSignInAsync(user.UserName,
25 model.Password, false, false);
26     if (result.IsLockedOut)
27         ModelState.AddModelError("", "Account geblokkeerd!!");
28
29     if (result.Succeeded)
30     {
31         return RedirectToAction("Index", "Home");
32     }
33
34     ModelState.AddModelError("", "Ongeldige loginpoging");
35     return View(model);
36 }
```

Beveiligen van de applicatie

In H6 en H7 heb je geleerd hoe je een applicatie kunt beveiligen met **[Authorize]** en **[Authorize(Roles = "Beheerder")]**. Dat is interessant als je niet wil dat bepaalde acties van je Controller door iedereen kunnen worden uitgevoerd. In MVC moet je echter nog een extra instelling doen in Program.cs. Je moet namelijk instellen wat er gebeurt als je naar een pagina navigeert waar je geen rechten op hebt. Voeg daarom volgende regels code toe:

```
1  builder.Services.ConfigureApplicationCookie(options =>                                CS
2  {
3      options.LoginPath = "/Gebruiker/Login"; // Pad naar de inlogpagina
4      als je niet aangemeld bent.
5      options.AccessDeniedPath = "/Home/Index"; // Pad naar de homepagina
6      als je aangemeld bent maar geen rechten hebt.
7  });
```

In een UI heb je nog een ander bijkomend probleem: Het is niet omdat bepaalde acties van een Controller beveiligd zijn, dat de bijhorende hyperlinks/knoppen op de View daarmee onzichtbaar zijn voor onbevoegde gebruikers. In de onderstaande afbeelding is de

Create, Edit en Delete actie beveiligd maar de knoppen zijn nog zichtbaar.

Bestellingen

[Nieuwe bestelling](#)

Id	Naam			
1	Leon Van Der Neffe	Details	Edit	Delete
2	Firmin Van De Kasseinen	Details	Edit	Delete
3	Marcel Kiekeboe	Details	Edit	Delete
4	Firmin Van De Kasseinen	Details	Edit	Delete
5	Marcel Kiekeboe	Details	Edit	Delete
6	Leon Van Der Neffe	Details	Edit	Delete

Daarvoor kun je in de View de knoppen gaan verbergen voor gebruikers zonder de juiste rol:

```
1  @{
2      if (User.IsInRole("Beheerder"))
3      {
4          <td><a asp-action="EditBestelling" asp-route-id="@item.Id"
5          class="btn btn-primary">Edit</a></td>
6          <td><a asp-action="DeleteBestelling" asp-route-id="@item.Id"
7          class="btn btn-primary">Delete</a></td>
8      }
9  }
```

Bestellingen

Id	Naam	
1	Leon Van Der Neffe	Details
2	Firmin Van De Kasseinen	Details
3	Marcel Kiekeboe	Details
4	Firmin Van De Kasseinen	Details
5	Marcel Kiekeboe	Details
6	Leon Van Der Neffe	Details

Gebruikersbeheer

Net zoals je voor bestellingen en klanten een indexpagina gemaakt hebt met een overzicht van alle records, kun je dat ook maken voor gebruikers of rollen. In de Index-actie kun je ook gebruikers afleiden naar een andere pagina als ze geen rechten hebben op het gebruikersbeheer.

```
1  [AllowAnonymous]
2  public IActionResult Index()
3  {
4      if (User.IsInRole("Gebruiker")) return RedirectToAction("Index",
5      "Home");
6
7      GebruikerListViewModel viewModel = new GebruikerListViewModel()
8      {
9          Gebruikers = _userManager.Users.ToList()
10     };
11     return View(viewModel);
12 }
```

Gebruikers aanmaken

In het laatste voorbeeld wordt nog het aanmaken van een nieuwe gebruiker besproken.

1. Maak een ViewModel `GebruikerCreateViewModel` . Je geeft een lijst mee van alle rollen en krijgt de Id terug van de geselecteerd rol.

```
1  public class GebruikerCreateViewModel
2  {
3      [Required]
4      public string GebruikersNaam { get; set; }
5      [Required]
6      public string Naam { get; set; }
7      [Required]
8      public string Voornaam { get; set; }
9      [Required]
10
11     public DateTime Geboortedatum { get; set; }
12     [Required]
```

```

12     public string Email { get; set; }
13     [DataType(DataType.Password)]
14     [Required]
15     public string Password { get; set; }
16     public SelectList? Rollen { get; set; }
17     public string RolId { get; set; }
18 }
19

```

2. Voorzie een actie in de `GebruikerController` om de lijst van rollen op te vullen

```

1 public IActionResult Create() CS
2 {
3     GebruikerCreateViewModel viewModel = new GebruikerCreateViewModel()
4     {
5         Geboortedatum = new DateTime(1950, 1, 1),
6         Rollen = new SelectList(_roleManager.Roles.ToList(), "Id",
7         "Name")
8     };
9     return View(viewModel);
10 }

```

3. Voorzie een View `Create` in de map `Views\Gebruiker`. Dit is vergelijkbaar met andere formulieren.

4. Maak een actie in de `GebruikerController` om de gebruiker aan te maken en daarna toe te wijzen aan de juiste rol.

```

1 [HttpPost] CS
2 [ValidateAntiForgeryToken]
3 public async Task<IActionResult> Create(GebruikerCreateViewModel
4 viewModel)
5 {
6     if (ModelState.IsValid)
7     {
8         CustomUser gebruiker = new CustomUser
9         {
10             Naam = viewModel.Naam,
11             Voornaam = viewModel.Voornaam,
12             Geboortedatum = viewModel.Geboortedatum,
13             UserName = viewModel.GebruikersNaam,
14             Email = viewModel.Email,
15             EmailConfirmed=true

```

```
16         };
17
18         IdentityResult result = await _userManager.CreateAsync(gebruiker,
19         viewModel.Password);
20
21         if (result.Succeeded)
22         {
23             CustomUser user = await
24             _userManager.FindByEmailAsync(viewModel.Email);
25             IdentityRole role = await
26             _roleManager.FindByIdAsync(viewModel.RoleId);
27
28             IdentityResult Resultaat = await
29             _userManager.AddToRoleAsync(user, role.Name);
30             return RedirectToAction("Index");
31         }
32         else
33         {
34             foreach (IdentityError error in result.Errors)
35                 ModelState.AddModelError("", error.Description);
36         }
37     }
38     return View(viewModel);
39 }
```

[Hier vind je het WebApiDemo-project.](#)

Laatst bijgewerkt: 09-09-2026, 08:52

Bijdragers: JorenSynaeveTM